DBMS & SQL

Teach-me-from-zero exam notes

Written for someone who knows a little SQL, has not practised, and has to **solve unseen questions** in the exam. Every SQL topic follows the same four beats: **(1) what it is, (2) the syntax pattern, (3) how to turn a question into the query, (4) the traps.** All SELECT outputs shown in green boxes were produced by actually running the query on the sample data, so you can trust them.

> **TRAP:** Reading these notes is not enough. SQL is a skill like cycling. For every worked example: cover the answer, write it on paper, then compare. Part 12 is a question bank for exactly that. If you only read, you will freeze in the exam.

Contents

Part	What is inside
0	The 6-step method for ANY SQL question + keyword-to-tool table (read this first)
1	The sample database used everywhere (memorise the shape of it)
2	DBMS basics: keys, integrity, DDL/DML/DQL/DCL/TCL, CREATE/INSERT/UPDATE/DELETE/ALTER
3	SELECT from zero: WHERE, LIKE, NULL, ORDER BY, LIMIT, DISTINCT, execution order
4	Aggregates, GROUP BY, HAVING
5	Joins: INNER, LEFT, RIGHT, self, 3-table, FULL, UNION
6	Subqueries: scalar, IN, correlated, EXISTS, ANY/ALL
7	CASE (full deep-dive, since you asked)
8	CTE (WITH) and window functions: ROW_NUMBER, RANK, DENSE_RANK, toppers, Nth highest
9	Views
10	Stored procedures, functions, triggers (templates to fill in)
11	Indexes, B-tree/B+ tree, assertions, query processing/optimization
12	SQL practice bank: 30 questions with solutions and thinking
13	Transactions, ACID, states
14	Schedules: conflict serializability, recoverability, 2PL, timestamps, deadlock, recovery
15	Theory answer frames + one-page cheat sheet

Part 0 - The 6-step method for any SQL question

Almost every exam SQL question can be solved by the same routine. Do not start typing. Do this on paper first:

1. **Underline what must appear in the output** ("display names and HOD names" -> output columns: Name, HOD). These go in SELECT.
2. **Find which table owns each column.** One table -> no join. Two or more -> you need a JOIN (step 4).
3. **Find the row filters** ("above 80", "in CSE", "starts with A") -> WHERE.
4. **If two tables:** find the shared column that links them (foreign key). That becomes the ON condition.
5. **Look for the words "each / per / for every / department-wise"** -> GROUP BY. Look for a condition on a group result ("more than 2 students", "average above 80") -> HAVING.
6. **Look for hidden two-stage logic:** "above average", "highest", "not in", "at least one", "rank", "top N per group" -> subquery / EXISTS / window function / CTE. Solve the inner stage on its own first, then wrap.

Keyword -> tool table (this is your decoder ring)

If the question says...	Think of...	Part
display / list / show / retrieve	SELECT ... FROM	3
only those / whose / having value / between / starting with	WHERE (+ LIKE, BETWEEN, IN)	3
highest first / sorted / alphabetical	ORDER BY ... DESC/ASC	3
top 3 / first 5	ORDER BY + LIMIT	3
total / average / count / max / min	Aggregate function	4
for each department / per course / department-wise	GROUP BY	4
departments having more than 2 / average above 80 (group condition)	HAVING	4
names with their HOD / student with course name	JOIN (INNER)	5
including those with no match / even if zero / all departments	LEFT JOIN (+ COUNT(right_col))	5
not enrolled / never ordered / without any	LEFT JOIN ... IS NULL or NOT EXISTS	5, 6
employee and their manager (same table twice)	SELF JOIN	5
above overall average / equal to maximum	Scalar subquery	6
above their own department average	Correlated subquery (or window)	6, 8
at least one / has any / exists	EXISTS	6
label as Pass/Fail / grade / category / band	CASE	7
count how many are Pass and how many Fail in one row	SUM(CASE ...) / COUNT(CASE ...)	7
every row plus its group average	Window function (AVG OVER PARTITION BY)	8
rank / topper per department / top N in each group / Nth highest	RANK/DENSE_RANK/ROW_NUMBER inside a CTE	8
save this query for reuse / hide columns	VIEW	9
store a routine and call it later	PROCEDURE (CALL)	10
return a calculated value (square, grade, percentage)	FUNCTION (RETURNS / RETURN)	10
automatically when a row is inserted/updated/deleted / log / audit / reject	TRIGGER	10

Order you write vs order SQL thinks (memorise this, it explains 90% of errors)

```
You WRITE:                        SQL EXECUTES in this order:
  SELECT  columns                   1. FROM / JOIN    (which tables, how matched)
  FROM    tables                    2. WHERE          (filter individual rows)
  WHERE   row-condition             3. GROUP BY       (make groups)
  GROUP BY columns                  4. HAVING         (filter groups)
  HAVING  group-condition           5. SELECT         (compute output columns, aliases born here)
  ORDER BY columns                  6. DISTINCT
  LIMIT   n                         7. ORDER BY       (can use aliases)
                                    8. LIMIT
```

REMEMBER: Because WHERE runs before SELECT: you cannot use a column alias or an aggregate inside WHERE. Because ORDER BY runs after SELECT: you CAN use an alias in ORDER BY. That single fact explains why topper-by-rank needs a CTE (Part 8).

Part 1 - The sample database (learn the shape)

All examples use one small college database, so you can concentrate on the SQL, not on new data each time. In the exam the table names will differ (Employee/Department, Customer/Orders...) - the **patterns** are identical.

Table	Columns	Rows
Department	DeptID (PK), Department (UNIQUE), HOD	CSE/Dr. Kumar, ECE/Dr. Priya, EEE/Dr. Ravi, ME/Dr. Anil
Student	StudentID (PK), Name, Department (FK), Marks, City	6 students
Course	CourseID (PK), CourseName, Credits	DBMS, Python, Data Structures, AI
Enrollment	StudentID, CourseID, Marks (PK = both)	8 rows. Links Student and Course
Employee	EmpID (PK), EmpName, Dept, Salary, ManagerID	extra table for salary/manager questions

```
SELECT * FROM Student
```

```
StudentID | Name  | Department | Marks | City
----------+-------+------------+-------+----------
101       | Anu   | CSE        | 85    | Bangalore
102       | Ravi  | CSE        | 72    | Chennai
103       | Priya | ECE        | 91    | Bangalore
104       | Kiran | EEE        | 65    | Hosur
105       | Meena | CSE        | 78    | Chennai
106       | Arjun | ECE        | 88    | Mysore
```

```
SELECT * FROM Enrollment
```

```
StudentID | CourseID | Marks
----------+----------+------
101       | C01      | 88
101       | C02      | 92
102       | C01      | 75
103       | C01      | 95
103       | C03      | 89
104       | C02      | 68
105       | C01      | 81
106       | C03      | 90
```

```
SELECT * FROM Employee
```

```
EmpID | EmpName | Dept  | Salary | ManagerID
------+---------+-------+--------+----------
1     | Kumar   | HR    | 50000  | NULL
2     | Asha    | IT    | 60000  | 1
3     | Ravi    | IT    | 60000  | 1
4     | Sita    | IT    | 75000  | 2
5     | John    | HR    | 45000  | 1
6     | Mary    | Sales | 55000  | 1
```

Facts to hold in your head (used for outer joins / NOT EXISTS answers)

- Department ME has **no students**. Course AI has **no enrollments**.
- Student.Marks = overall marks. Enrollment.Marks = marks in ONE course. Same column name, different meaning. Read which one the question asks.
- Employee.ManagerID points back to Employee.EmpID - that is the self-join setup. Kumar has no manager (NULL).

Part 2 - DBMS basics and the data-definition/manipulation commands

2.1 What is a DBMS / RDBMS (definition-type answers)

- **Database** = organised collection of related data. **DBMS** = software that stores, retrieves and controls it. **RDBMS** = stores data in related **tables** (relations). Examples: MySQL, Oracle, PostgreSQL, SQL Server. Relational model: E. F. Codd, 1970.
- **Relation** = table, **tuple** = row, **attribute** = column, **domain** = set of allowed values of a column. **Schema** = design; **instance** = data at a moment.
- Benefits: less redundancy, integrity via constraints, shared multi-user access, security, backup/recovery, transactions. Limits: costly joins at huge scale, schema changes are heavy, admin overhead.
- Architecture: query processor (parser -> optimizer -> executor), transaction manager, storage manager, data dictionary (metadata). Physical independence: change storage/indexes without changing programs. Logical independence: some schema changes hidden by views.

2.2 Keys and integrity (very common 2-5 mark question)

Term	Meaning	Example
Super key	Any set of columns that uniquely identifies a row	{StudentID}, {StudentID, Name}
Candidate key	Minimal super key (no column can be removed)	StudentID
Primary key	The candidate key you chose. Unique and NOT NULL. One per table	StudentID
Alternate key	Candidate keys not chosen as primary	Email, if unique
Composite key	Key made of 2+ columns	Enrollment(StudentID, CourseID)
Foreign key	Column(s) referring to the key of another (or same) table	Student.Department -> Department.Department
Entity integrity	Primary key is unique and not NULL	
Referential integrity	A non-NULL foreign key value must exist in the parent table	Cannot insert a student into dept "XYZ"
Domain integrity	Values obey type and rules	Marks between 0 and 100

2.3 The five SQL command groups

Group	Full form	Purpose	Commands
DDL	Data Definition Language	Create/alter structure	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	Change data	INSERT, UPDATE, DELETE
DQL	Data Query Language	Read data	SELECT
DCL	Data Control Language	Permissions	GRANT, REVOKE
TCL	Transaction Control Language	Save/undo work	COMMIT, ROLLBACK, SAVEPOINT, START TRANSACTION

2.4 CREATE TABLE - how to build it from a requirement

Pattern: table name, then inside brackets a list of *column name + type + constraints*, separated by commas (no comma after the last one), then constraints that involve several columns.

```
CREATE TABLE table_name (
    column1  datatype  [constraints],
    column2  datatype  [constraints],
    ...,
    PRIMARY KEY (col),                          -- or inline
    FOREIGN KEY (col) REFERENCES parent(col)
);
```

Type	Stores	Note
INT	whole number	
DECIMAL(5,2)	exact number, 5 digits total, 2 after the point	max 999.99
VARCHAR(50)	text up to 50 chars	single quotes for values: 'CSE'
DATE / TIMESTAMP	date / date+time	'2025-03-01'

Constraint	What it does
`PRIMARY KEY`	unique + NOT NULL, identifies row
`NOT NULL`	value is compulsory
`UNIQUE`	no duplicates (NULLs allowed)
`DEFAULT x`	value used when you omit it
`CHECK (cond)`	value must satisfy condition, e.g. CHECK (Marks BETWEEN 0 AND 100)
`FOREIGN KEY ... REFERENCES`	must match a parent key. Parent table must exist first
`AUTO_INCREMENT`	MySQL: auto-generates 1,2,3... for ids

Worked requirement: "Create Student with unique ID, compulsory name, marks 0-100, and department referencing Department."

```
CREATE TABLE Student (
    StudentID  INT PRIMARY KEY,
    Name       VARCHAR(100) NOT NULL,
    Department VARCHAR(50)  NOT NULL,
    Marks      DECIMAL(5,2) CHECK (Marks BETWEEN 0 AND 100),
    City       VARCHAR(50),
    FOREIGN KEY (Department) REFERENCES Department(Department)
);
```

TIP: Create the PARENT table first (Department before Student; Student and Course before Enrollment). Insert parent rows before child rows.

2.5 INSERT, UPDATE, DELETE, ALTER, DROP

```
INSERT INTO Student (StudentID, Name, Department, Marks, City)
VALUES (107, 'Dev', 'CSE', 75, 'Chennai');          -- one row
INSERT INTO Student VALUES (108,'Zoya','ECE',60,'Mysore'), (109,'Om','EEE',55,'Hosur');  -- several

UPDATE Student SET Marks = Marks + 5 WHERE StudentID = 102;   -- WHERE = which rows
UPDATE Student SET Marks = 90, City = 'Pune' WHERE Name = 'Ravi';  -- many columns

DELETE FROM Enrollment WHERE StudentID = 102;   -- delete child rows first (FK!)
DELETE FROM Student    WHERE StudentID = 102;
```

TRAP: UPDATE or DELETE **without WHERE** affects EVERY row. Exam answers lose marks for this.

Command	Removes	Notes
`DELETE FROM t WHERE ...`	chosen rows	DML, can be rolled back, supports WHERE
`TRUNCATE TABLE t`	all rows	DDL, no WHERE, fast, resets auto-increment, cannot rollback in MySQL
`DROP TABLE t`	table itself (structure + data)	DDL, gone completely

```
ALTER TABLE Student ADD Email VARCHAR(100);           -- add a column
ALTER TABLE Student MODIFY Email VARCHAR(150);        -- change type/size
ALTER TABLE Student DROP COLUMN Email;                -- remove column
ALTER TABLE Student RENAME COLUMN City TO Town;       -- rename (MySQL 8)
DESCRIBE Student;     SHOW TABLES;
GRANT SELECT ON CollegeDB.Student TO 'learner'@'localhost';   -- DCL
REVOKE SELECT ON CollegeDB.Student FROM 'learner'@'localhost';
START TRANSACTION;  UPDATE ...;  SAVEPOINT s1;  UPDATE ...;  ROLLBACK TO s1;  COMMIT;   -- TCL
```

Part 3 - SELECT from zero

3.1 The three-word skeleton

```
SELECT  what-to-show        -- columns, * for all, or calculations
FROM    where-it-lives      -- table
WHERE   which-rows;         -- optional filter
```

Read it in English: "**Show** these columns, **from** this table, **keeping only** rows where this is true."

```
SELECT Name, Marks FROM Student WHERE Marks > 80
```

```
Name  | Marks
------+------
Anu   | 85
Priya | 91
Arjun | 88
```

Question -> query in 4 moves. "Display names and cities of CSE students": (1) output: Name, City -> SELECT Name, City. (2) table holding them: Student -> FROM Student. (3) restriction: CSE -> WHERE Department = 'CSE'. (4) semicolon.

```
SELECT Name, City FROM Student WHERE Department = 'CSE'
```

```
Name  | City
------+----------
Anu   | Bangalore
Ravi  | Chennai
Meena | Chennai
```

Small variations you must know

```
SELECT DISTINCT Department FROM Student
```

```
Department
----------
CSE
ECE
EEE
```

```
SELECT Name, Marks, Marks + 5 AS BonusMarks FROM Student
```

```
Name  | Marks | BonusMarks
------+-------+-----------
Anu   | 85    | 90
Ravi  | 72    | 77
Priya | 91    | 96
Kiran | 65    | 70
Meena | 78    | 83
Arjun | 88    | 93
```

- `AS` gives a column (or table) a nickname. It changes the display only, never the stored data.
- `DISTINCT` removes duplicate output rows.
- `SELECT *` means all columns.
- Text values in single quotes: 'CSE'. Column/table names have NO quotes.

3.2 WHERE - writing the condition

Question wording	Condition
above 80 / at least 80	Marks > 80 / Marks >= 80
below 80 / at most 80	Marks < 80 / Marks <= 80
equal / not equal	Marks = 80 / Marks <> 80 (also !=)
from 70 to 85 inclusive	Marks BETWEEN 70 AND 85
CSE or ECE	Department IN ('CSE','ECE')

Question wording	Condition
not CSE or ECE	Department NOT IN ('CSE','ECE')
name starts with A / ends with a / contains ri	Name LIKE 'A%' / '%a' / '%ri%'
name is exactly 4 letters	Name LIKE '____' (underscore = exactly one character)
city is missing	City IS NULL (never = NULL)
city is present	City IS NOT NULL

```
SELECT Name, Marks FROM Student WHERE (Department='CSE' OR Department='ECE') AND Marks >= 85
```

```
Name  | Marks
------+------
Anu   | 85
Priya | 91
Arjun | 88
```

TRAP: AND binds tighter than OR. "CSE or ECE students with marks >= 85" needs **brackets** around the OR. Without them SQL reads it as CSE OR (ECE AND marks>=85).

TRAP: NULL is "unknown". `City = NULL` is never true. Use `IS NULL`. Also NULL <> anything gives unknown, so rows with NULL silently disappear from comparisons.

3.3 ORDER BY, LIMIT, top-N

```
SELECT Name, Marks FROM Student ORDER BY Marks DESC, Name ASC LIMIT 3
```

```
Name  | Marks
------+------
Priya | 91
Arjun | 88
Anu   | 85
```

- ASC = low to high (default). DESC = high to low. Several keys: ties on the first are broken by the second.
- LIMIT 3 = first 3 rows after sorting. "Top 3" = ORDER BY ... DESC LIMIT 3. LIMIT 2, 3 (or LIMIT 3 OFFSET 2) = skip 2 rows, then take 3.
- LIMIT does NOT include ties automatically. If tied people must all be shown, use a subquery or RANK (Part 8).

3.4 Useful scalar functions

Function	Example -> result
`UPPER / LOWER`	UPPER('anu') -> ANU
`LENGTH`	LENGTH('Anu') -> 3
`CONCAT`	CONCAT(Name,' - ',City)
`ROUND(x,d)`	ROUND(79.8333,2) -> 79.83
`COALESCE(a,b)`	COALESCE(City,'Unknown') -> replaces NULL
`CURDATE() / NOW()`	current date / datetime

Part 4 - Aggregates, GROUP BY, HAVING

4.1 Aggregate functions: many rows in, ONE value out

```
SELECT COUNT(*) AS n, SUM(Marks) AS total, AVG(Marks) AS average, MAX(Marks) AS highest,
    MIN(Marks) AS lowest
FROM Student
```

```
n | total | average | highest | lowest
--+-------+---------+---------+-------
6 | 479   | 79.83   | 91      | 65
```

- COUNT(*) counts rows. COUNT(col) counts rows where col is NOT NULL. COUNT(DISTINCT col) counts unique values.
- SUM, AVG, MAX, MIN ignore NULLs. If all inputs are NULL they return NULL (COUNT returns 0).
- You cannot mix a plain column with an aggregate unless that column is in GROUP BY: `SELECT Name, MAX(Marks)` is wrong/meaningless. "Name of the highest scorer" needs a subquery (Part 6).

4.2 GROUP BY: one result per group

Use it when the question says **each / per / for every / department-wise**. SQL splits rows into piles by the GROUP BY column, then applies the aggregate to each pile.

```
SELECT Department, COUNT(*) AS students, AVG(Marks) AS avg_marks
FROM Student
GROUP BY Department
```

```
Department | students | avg_marks
-----------+----------+----------
CSE        | 3        | 78.33
ECE        | 2        | 89.5
EEE        | 1        | 65
```

Golden rule: every column in SELECT must either be in GROUP BY or be inside an aggregate function.

4.3 HAVING: filter the groups

```
SELECT Department, COUNT(*) AS students FROM Student GROUP BY Department HAVING COUNT(*) > 1
```

```
Department | students
-----------+---------
CSE        | 3
ECE        | 2
```

WHERE	HAVING
filters individual rows	filters groups
runs BEFORE grouping	runs AFTER grouping
cannot contain aggregates	normally contains an aggregate
WHERE Marks >= 80	HAVING COUNT(*) > 1 / HAVING AVG(Marks) > 80

Both together: "departments with at least 2 students scoring 80 or more". WHERE removes weak students first, then groups are counted, then HAVING filters groups.

```
SELECT Department, COUNT(*) AS high_scorers
FROM Student
WHERE Marks >= 80
GROUP BY Department
HAVING COUNT(*) >= 2
```

```
Department | high_scorers
-----------+-------------
ECE        | 2
```

Template for any "per group + condition" question

```
SELECT  group_col, AGG(...)
FROM    table
WHERE   row-conditions            -- optional
GROUP BY group_col
HAVING  AGG(...) condition        -- optional
ORDER BY ... ;                    -- optional
```

Classic: find duplicates. "Names that appear more than once" -> group by the thing that repeats, keep groups with count > 1:

```
SELECT Salary, COUNT(*) AS how_many FROM Employee GROUP BY Salary HAVING COUNT(*) > 1
```

```
Salary | how_many
-------+---------
60000  | 2
```

Classic: department with the highest average (use ORDER BY + LIMIT 1):

```
SELECT Department, AVG(Marks) AS avg_marks
FROM Student
GROUP BY Department
ORDER BY avg_marks DESC
LIMIT 1
```

```
Department | avg_marks
-----------+----------
ECE        | 89.5
```

Part 5 - Joins

5.1 Why joins exist

Data is split across tables to avoid repetition. Student knows the department *name*; only Department knows the *HOD*. To show Name beside HOD you must **match rows** from the two tables using the shared column. That matching is a join.

5.2 INNER JOIN - only rows that match on both sides

```
SELECT   s.Name, d.HOD
FROM     Student s                          -- s = nickname (alias) for Student
INNER JOIN Department d                     -- d = nickname for Department
         ON s.Department = d.Department;    -- the matching rule
```

```
SELECT s.Name, d.HOD FROM Student s INNER JOIN Department d ON s.Department = d.Department
```

```
Name  | HOD
------+----------
Anu   | Dr. Kumar
Ravi  | Dr. Kumar
Priya | Dr. Priya
Kiran | Dr. Ravi
Meena | Dr. Kumar
Arjun | Dr. Priya
```

- **ON** = how rows are matched (almost always foreign key = primary key). **WHERE** = filter after matching.
- The dot (`s.Name`) says which table a column comes from. Needed when both tables have a column of the same name.
- Forgetting ON gives a cartesian product (every row x every row) - wrong and huge.
- JOIN alone = INNER JOIN.

5.3 LEFT / RIGHT JOIN - keep unmatched rows

Trigger words: **"all", "including those with no ...", "even if zero", "not enrolled", "without any"**. INNER JOIN would drop the unmatched rows; LEFT JOIN keeps every row of the LEFT table and fills NULL where the right side has no match.

```
SELECT d.Department, s.Name
FROM Department d
  LEFT
  JOIN Student s ON d.Department = s.Department
```

```
Department | Name
-----------+------
CSE        | Anu
CSE        | Meena
CSE        | Ravi
ECE        | Arjun
ECE        | Priya
EEE        | Kiran
ME         | NULL
```

Counting with zero: use COUNT(right_table.key), NOT COUNT(*). COUNT(*) would count the NULL-filled row for ME as 1.

```
SELECT d.Department, COUNT(s.StudentID) AS students
FROM Department d
  LEFT
  JOIN Student s ON d.Department = s.Department
GROUP BY d.Department
```

```
Department | students
-----------+---------
CSE        | 3
ECE        | 2
EEE        | 1
ME         | 0
```

Finding the unmatched: "Departments with no students" = LEFT JOIN then keep rows where the right key IS NULL.

```
SELECT d.Department
FROM Department d
  LEFT
  JOIN Student s ON d.Department = s.Department
WHERE s.StudentID IS NULL
```

```
Department
----------
ME
```

```
SELECT c.CourseName
FROM Course c
  LEFT
  JOIN Enrollment e ON c.CourseID = e.CourseID
WHERE e.StudentID IS NULL
```

```
CourseName
----------
AI
```

TRAP: Putting a filter on the right table in WHERE (e.g. WHERE s.Marks >= 80) after a LEFT JOIN deletes the NULL rows and turns it back into an inner join. If the filter only defines which optional rows may match, put it in the ON clause: ... ON d.Department = s.Department AND s.Marks >= 80.

RIGHT JOIN is the mirror image: it keeps all rows of the right table. `A LEFT JOIN B` is the same as `B RIGHT JOIN A`. Most people just use LEFT and swap table order.

5.4 Three-table join (many-to-many through a bridge table)

Student and Course are linked only through Enrollment. Walk the chain: Student -> Enrollment -> Course. Each JOIN needs its own ON.

```
SELECT s.Name, c.CourseName, e.Marks
FROM Student s
  JOIN Enrollment e ON s.StudentID = e.StudentID
  JOIN Course c ON e.CourseID = c.CourseID
WHERE c.CourseName = 'DBMS'
ORDER BY e.Marks DESC
```

```
Name  | CourseName | Marks
------+------------+------
Priya | DBMS       | 95
Anu   | DBMS       | 88
Meena | DBMS       | 81
Ravi  | DBMS       | 75
```

TIP: For N tables you need N-1 join conditions. Draw boxes and arrows (foreign keys) first, then write the joins following the arrows.

5.5 Self join - a table joined with itself

Use when rows of a table refer to other rows of the same table (employee -> manager). Treat it as **two copies with different aliases**.

```
SELECT e.EmpName AS Employee, m.EmpName AS Manager
FROM Employee e
  LEFT
  JOIN Employee m ON e.ManagerID = m.EmpID
```

```
Employee | Manager
---------+--------
Kumar    | NULL
Asha     | Kumar
Ravi     | Kumar
Sita     | Asha
John     | Kumar
Mary     | Kumar
```

Pairs in the same department (the < removes self-pairs and mirror duplicates):

```
SELECT a.Name, b.Name
FROM Student a
  JOIN Student b ON a.Department = b.Department
WHERE a.StudentID < b.StudentID
```

```
Name  | Name
------+------
Anu   | Ravi
Anu   | Meena
Ravi  | Meena
Priya | Arjun
```

Employees earning more than their manager: `SELECT e.EmpName FROM Employee e JOIN Employee m ON e.ManagerID = m.EmpID WHERE e.Salary > m.Salary;`

5.6 FULL OUTER JOIN and UNION

- FULL OUTER JOIN = keep unmatched rows from BOTH sides. **MySQL does not have it.** Emulate: LEFT JOIN **UNION** RIGHT JOIN.
- **UNION** stacks results of two queries and removes duplicates. **UNION ALL** keeps duplicates. Both queries need the same number of columns with compatible types.
- JOIN adds columns side by side (combines related info). UNION adds rows on top of each other.

```
SELECT d.Department, s.Name FROM Department d LEFT  JOIN Student s ON d.Department = s.Department
UNION
SELECT d.Department, s.Name FROM Department d RIGHT JOIN Student s ON d.Department = s.Department;
```

Join	Returns
INNER	only matching pairs
LEFT	all left rows + matches (NULL if none)
RIGHT	all right rows + matches
FULL (UNION emulation)	all rows of both
CROSS	every combination (no ON)
SELF	table joined to itself with 2 aliases

Part 6 - Subqueries

6.1 What and when

A **subquery** is a SELECT inside another statement, wrapped in brackets. Use it when you must **first calculate something** and then compare against it. The tell-tale words: **above average, equal to the maximum, same as, in the list of, not in, at least one, no match**.

> **TIP:** Technique: write the INNER query alone and check it works. Then paste it in brackets into the outer query. Never write both at once.

6.2 Scalar subquery (inner returns ONE value) - use = > < etc.

"Students scoring above the overall average." Stage 1: `SELECT AVG(Marks) FROM Student` -> 79.83. Stage 2: plug it in.

```
SELECT Name, Marks FROM Student WHERE Marks > (SELECT AVG(Marks) FROM Student)
```

```
Name  | Marks
------+------
Anu   | 85
Priya | 91
Arjun | 88
```

"Name of the highest scorer" (fixes the MAX-with-Name problem):

```
SELECT Name, Marks FROM Student WHERE Marks = (SELECT MAX(Marks) FROM Student)
```

```
Name  | Marks
------+------
Priya | 91
```

Second highest mark (very common): the max of everything below the max.

```
SELECT MAX(Marks) AS second_highest
FROM Student
WHERE Marks < (SELECT MAX(Marks)
FROM Student)
```

```
second_highest
--------------
88
```

```
SELECT Name, Marks
FROM Student
WHERE Marks = (SELECT MAX(Marks)
FROM Student
WHERE Marks < (SELECT MAX(Marks)
FROM Student))
```

```
Name  | Marks
------+------
Arjun | 88
```

6.3 IN / NOT IN (inner returns a LIST)

```
SELECT Name
FROM Student
WHERE StudentID IN (SELECT StudentID
FROM Enrollment
WHERE CourseID = 'C01')
```

```
Name
-----
Anu
Ravi
Priya
Meena
```

```
SELECT Name
FROM Student
WHERE StudentID NOT IN (SELECT StudentID
FROM Enrollment
WHERE CourseID = 'C01')
```

```
Name
-----
Kiran
Arjun
```

TRAP: If the inner list contains even one NULL, NOT IN returns **no rows** (comparison with NULL is unknown). Prefer NOT EXISTS for "has no match".

6.4 Correlated subquery - the inner query depends on the current outer row

"Students scoring above **their own department's** average." The average is different for every department, so the inner query must know which department the outer row belongs to.

```
SELECT s.Name, s.Department, s.Marks
FROM   Student s                      -- OUTER row = the student being tested
WHERE  s.Marks > (
         SELECT AVG(t.Marks)
         FROM   Student t             -- second copy of the table
         WHERE  t.Department = s.Department   -- <-- the "correlation": link inner to outer
       );
```

```
SELECT s.Name, s.Department, s.Marks
FROM Student s
WHERE s.Marks > (SELECT AVG(t.Marks)
FROM Student t
WHERE t.Department = s.Department)
```

```
Name  | Department | Marks
------+------------+------
Anu   | CSE        | 85
Priya | ECE        | 91
```

Outer student	Dept average computed by inner query	Keep?
Anu 85 (CSE)	(85+72+78)/3 = 78.33	yes
Ravi 72 (CSE)	78.33	no
Meena 78 (CSE)	78.33	no
Priya 91 (ECE)	(91+88)/2 = 89.5	yes
Arjun 88 (ECE)	89.5	no (88 < 89.5)
Kiran 65 (EEE)	65	no (equal is not greater)

REMEMBER: Your original slides wrongly list Arjun here. ECE average is 89.5, Arjun has 88, so he does NOT qualify. Correct answer: Anu, Priya.

Reading the correlated line: "inside the average, include only students whose department equals the department of the student I am currently checking." Same pattern for "topper in each department": change AVG to MAX and > to =.

```
SELECT s.Name, s.Department, s.Marks
FROM Student s
WHERE s.Marks = (SELECT MAX(t.Marks)
FROM Student t
WHERE t.Department = s.Department)
```

```
Name  | Department | Marks
------+------------+------
Anu   | CSE        | 85
Priya | ECE        | 91
Kiran | EEE        | 65
```

6.5 EXISTS / NOT EXISTS - "is there at least one matching row?"

EXISTS is true if the inner query returns **any** row. What the inner SELECT lists does not matter (people write SELECT 1). You nearly always correlate it.

```
SELECT s.Name
FROM Student s
WHERE EXISTS (SELECT 1
FROM Enrollment e
WHERE e.StudentID = s.StudentID AND e.CourseID = 'C02')
```

```
Name
-----
Anu
Kiran
```

NOT EXISTS = "has no matching row" - the safe way to write "never / without any / not enrolled".

```
SELECT c.CourseName
FROM Course c
WHERE NOT EXISTS (SELECT 1
FROM Enrollment e
WHERE e.CourseID = c.CourseID)
```

```
CourseName
----------
AI
```

6.6 ANY and ALL

Phrase	Meaning	Shortcut (non-empty list, no NULL)
x > ANY (list)	bigger than at least one value	x > MIN(list)
x > ALL (list)	bigger than every value	x > MAX(list)
x = ANY (list)	equals at least one	same as IN
x <> ALL (list)	differs from every value	same as NOT IN

```
SELECT Name, Marks
FROM Student
WHERE Marks > ALL (SELECT Marks
FROM Student
WHERE Department = 'CSE')
```

```
Name  | Marks
------+------
Priya | 91
Arjun | 88
```

```
SELECT Name, Marks
FROM Student
WHERE Marks > ANY (SELECT Marks
FROM Student
WHERE Department = 'CSE')
```

```
Name  | Marks
------+------
Anu   | 85
Priya | 91
Meena | 78
Arjun | 88
```

Which subquery type? (decision flow)

```
Inner query returns ONE value (avg, max)?          -> compare with  =  >  <
Inner query returns a LIST of values?              -> IN / NOT IN / ANY / ALL
Question is only "does a related row exist?"       -> EXISTS / NOT EXISTS
Comparison value changes per outer row ("own")?    -> correlated (link inner to outer in WHERE)
```

Part 7 - CASE (from absolute zero)

7.1 What CASE is

CASE is SQL's if / else-if / else, but it lives inside a query and produces a value for each row. You use it whenever the question says: label, categorise, grade, classify, "show Pass or Fail", "Low/Medium/High", or count rows by category.

> **REMEMBER:** CASE is an **expression** (it returns a value), not a statement. So it goes wherever a value could go: in SELECT, ORDER BY, WHERE, GROUP BY, or inside SUM/COUNT. It is NOT the same as the IF ... END IF used inside stored procedures (Part 10).

7.2 Syntax - the searched form (use this 95% of the time)

```
CASE
    WHEN condition1 THEN result1
    WHEN condition2 THEN result2
    ...
    ELSE default_result
END
```

Word	Plain meaning
`CASE`	start of the decision
`WHEN condition`	test this (any condition you could put in WHERE)
`THEN result`	if the test is true, the answer for this row is this value
`ELSE result`	if nothing matched use this (optional; if omitted and nothing matches you get NULL)
`END`	close the CASE (compulsory; forgetting it is the classic syntax error)
`AS name`	give the new column a name - put it AFTER END

7.3 Build one step by step

Question: "Show each student's name, marks and Pass/Fail. Pass means marks at least 75."

1. Start with the plain part: `SELECT Name, Marks FROM Student`.
2. The extra column is a new output column, so add a comma and open `CASE`.
3. Translate the condition: "at least 75" -> `Marks >= 75`. Write `WHEN Marks >= 75 THEN 'Pass'`.
4. Everything else fails: `ELSE 'Fail'`. Close with `END AS Result`.

```
SELECT Name, Marks, CASE WHEN Marks >= 75 THEN 'Pass' ELSE 'Fail' END AS Result FROM Student
```

```
Name  | Marks | Result
------+-------+-------
Anu   | 85    | Pass
Ravi  | 72    | Fail
Priya | 91    | Pass
Kiran | 65    | Fail
Meena | 78    | Pass
Arjun | 88    | Pass
```

7.4 Several branches (grades): ORDER OF WHENs MATTERS

SQL checks WHENs **top to bottom and stops at the first true one**. So for overlapping ranges put the strictest (highest) threshold first.

```
SELECT Name, Marks, CASE WHEN Marks >= 90 THEN 'A' WHEN Marks >= 80 THEN 'B' WHEN Marks >=
    70 THEN 'C' ELSE 'D' END AS Grade
FROM Student
ORDER BY Marks DESC
```

```
Name  | Marks | Grade
------+-------+------
Priya | 91    | A
Arjun | 88    | B
Anu   | 85    | B
Meena | 78    | C
Ravi  | 72    | C
Kiran | 65    | D
```

TRAP: If you wrote WHEN Marks >= 70 first, then 91 would get C (first true branch wins) and the A/B branches would never fire. Alternatively write non-overlapping ranges with BETWEEN: WHEN Marks BETWEEN 80 AND 89 THEN ... (but beware gaps for decimals like 89.5).

7.5 The simple form (only for equality against one column)

```
CASE column
    WHEN value1 THEN result1
    WHEN value2 THEN result2
    ELSE other
END
```

```
SELECT Name, CASE Department WHEN 'CSE' THEN 'Computer' WHEN 'ECE' THEN 'Electronics' ELSE
    'Other' END AS Branch
FROM Student
```

```
Name  | Branch
------+------------
Anu   | Computer
Ravi  | Computer
Priya | Electronics
Kiran | Other
Meena | Computer
Arjun | Electronics
```

It can only test **=**. For >, <, BETWEEN, LIKE, IS NULL you must use the searched form.

7.6 CASE inside aggregates - "count by category" in one row (high-value exam pattern)

Question: "Show how many students passed (>=75) and how many failed, in a single row." CASE turns each row into 1 or 0, then SUM adds them.

```
SELECT SUM(CASE WHEN Marks >= 75 THEN 1 ELSE 0 END) AS passed, SUM(CASE WHEN Marks < 75 THEN
    1 ELSE 0 END) AS failed
FROM Student
```

```
passed | failed
-------+-------
4      | 2
```

Same idea per group - department-wise pass/fail counts:

```
SELECT Department, COUNT(CASE WHEN Marks >= 75 THEN 1 END) AS passed, COUNT(CASE WHEN Marks
    < 75 THEN 1 END) AS failed
FROM Student
GROUP BY Department
```

```
Department | passed | failed
-----------+--------+-------
CSE        | 2      | 1
ECE        | 2      | 0
EEE        | 0      | 1
```

`COUNT(CASE WHEN ... THEN 1 END)` works because with no ELSE the non-matching rows give NULL, and COUNT ignores NULL. Both forms are fine; SUM(CASE ... 1 ELSE 0) is easier to reason about.

7.7 Other places CASE appears

```
-- (a) in ORDER BY: custom sort, put CSE first then everyone else
SELECT Name, Department FROM Student
ORDER BY CASE WHEN Department = 'CSE' THEN 0 ELSE 1 END, Name;

-- (b) in UPDATE: different raise for different people (STORES the result)
UPDATE Employee
SET    Salary = CASE WHEN Salary < 50000 THEN Salary * 1.10
                     ELSE Salary * 1.05 END;

-- (c) NULL handling
SELECT Name, CASE WHEN City IS NULL THEN 'Unknown' ELSE City END AS City FROM Student;
```

CASE checklist

- Every WHEN has THEN. The last token before the alias is END.
- Text results in single quotes; all THEN/ELSE values should be the same kind (all text or all numbers).
- Highest/strictest condition first. Add ELSE or you may get NULL.
- In SELECT, CASE only **displays**; it changes stored data only inside UPDATE.
- Comma between CASE column and the previous column; none after it before FROM.

Part 8 - CTE and window functions

8.1 CTE (WITH ... AS) - give an intermediate result a name

A CTE (Common Table Expression) is a temporary named result that exists **only for the one statement** that follows it. Use it when the answer needs two stages: calculate something, then filter/query that calculation.

```
WITH name AS (
    SELECT ...            -- stage 1, any query
)
SELECT ...                -- stage 2, treats name like a table
FROM   name
WHERE  ...;               -- note: NO semicolon between the two stages
```

"Departments whose average is above 80":

```
WITH dept_avg AS (SELECT Department, AVG(Marks) AS avg_marks
FROM Student
GROUP BY Department) SELECT Department, avg_marks
FROM dept_avg
WHERE avg_marks > 80
```

```
Department | avg_marks
-----------+----------
ECE        | 89.5
```

- Inside the CTE the average is an ordinary column, so the outer filter is WHERE (not HAVING).
- View vs CTE: a view is saved in the database permanently; a CTE lives for one statement only.
- Recursive CTE (mentioned in slides): `WITH RECURSIVE n AS (SELECT 1 AS x UNION ALL SELECT x+1 FROM n WHERE x < 3) SELECT x FROM n;` -> 1,2,3. Needs: a starting row, a repeated step, a stopping condition. Used for hierarchies.

8.2 Window functions - calculate across rows WITHOUT collapsing them

Problem: "Show every student with the average of *their department* beside them." GROUP BY would give one row per department and lose the students. A window function keeps every row.

```
function(...) OVER ( [PARTITION BY group_column]  [ORDER BY sort_column] )
                      ^ split into groups          ^ order inside each group
```

```
SELECT Name, Department, Marks, AVG(Marks) OVER (PARTITION BY Department) AS dept_avg
FROM Student
ORDER BY Department, Marks DESC
```

```
Name  | Department | Marks | dept_avg
------+------------+-------+---------
Anu   | CSE        | 85    | 78.33
Meena | CSE        | 78    | 78.33
Ravi  | CSE        | 72    | 78.33
Priya | ECE        | 91    | 89.5
Arjun | ECE        | 88    | 89.5
Kiran | EEE        | 65    | 65
```

- **GROUP BY** = collapses rows to one per group. **PARTITION BY** = defines groups for the calculation but keeps all rows.
- `OVER ()` with empty brackets = whole table is one group.
- Window functions can be used only in SELECT / ORDER BY, not in WHERE.

8.3 Ranking: ROW_NUMBER, RANK, DENSE_RANK

Marks	ROW_NUMBER()	RANK()	DENSE_RANK()
95	1	1	1
90	2	2	2
90	3	2	2

Marks	ROW_NUMBER()	RANK()	DENSE_RANK()
80	4	4 (gap!)	3 (no gap)

- **ROW_NUMBER**: always 1,2,3,4. Ties get different numbers (arbitrary unless you add a tie-breaker in ORDER BY).
- **RANK**: ties share a rank, next rank skips (1,2,2,4).
- **DENSE_RANK**: ties share a rank, no skipping (1,2,2,3).

```
SELECT Name, Department, Marks, RANK() OVER (PARTITION BY Department ORDER BY Marks DESC) AS
    dept_rank
FROM Student
ORDER BY Department, dept_rank
```

```
Name  | Department | Marks | dept_rank
------+------------+-------+----------
Anu   | CSE        | 85    | 1
Meena | CSE        | 78    | 2
Ravi  | CSE        | 72    | 3
Priya | ECE        | 91    | 1
Arjun | ECE        | 88    | 2
Kiran | EEE        | 65    | 1
```

8.4 THE topper / top-N-per-group pattern (memorise this skeleton)

You cannot write `WHERE dept_rank = 1` in the same query that defines it (WHERE runs before windows exist). Calculate the rank in a CTE, filter outside.

```
WITH ranked AS (
    SELECT Name, Department, Marks,
           RANK() OVER (PARTITION BY Department ORDER BY Marks DESC) AS rnk
    FROM   Student
)
SELECT Name, Department, Marks FROM ranked WHERE rnk = 1;     -- change 1 to N for top N
```

```
WITH ranked AS (SELECT Name, Department, Marks, RANK() OVER (PARTITION BY Department ORDER
    BY Marks DESC) AS rnk
FROM Student) SELECT Name, Department, Marks
FROM ranked
WHERE rnk = 1
```

```
Name  | Department | Marks
------+------------+------
Anu   | CSE        | 85
Priya | ECE        | 91
Kiran | EEE        | 65
```

Choosing the function depends on the wording:

Wording	Use	Filter
exactly 2 per department (ties broken arbitrarily)	ROW_NUMBER (add a tie-breaker like StudentID)	rn <= 2
everyone in the top 2 score levels	DENSE_RANK	rnk <= 2
everyone whose competition rank is at most 2	RANK	rnk <= 2
the topper(s)	RANK (or DENSE_RANK)	rnk = 1

```
WITH r AS (SELECT Name, Department, Marks, ROW_NUMBER() OVER (PARTITION BY Department ORDER
    BY Marks DESC, StudentID) AS rn
FROM Student) SELECT Name, Department, Marks
FROM r
WHERE rn <= 2
```

```
Name  | Department | Marks
------+------------+------
Anu   | CSE        | 85
Meena | CSE        | 78
Priya | ECE        | 91
Arjun | ECE        | 88
Kiran | EEE        | 65
```

8.5 Nth highest (asked in almost every SQL course)

Third-highest distinct salary from Employee. Three ways - know at least two:

Way 1: ORDER BY + LIMIT/OFFSET (quickest; DISTINCT handles ties)

```
SELECT DISTINCT Salary FROM Employee ORDER BY Salary DESC LIMIT 1 OFFSET 2
```

```
Salary
------
55000
```

Way 2: DENSE_RANK (best: also returns the people)

```
WITH r AS (SELECT EmpName, Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk
FROM Employee) SELECT EmpName, Salary
FROM r
WHERE rnk = 3
```

```
EmpName | Salary
--------+-------
Mary    | 55000
```

Way 3: correlated count (old-school, no window functions)

```
SELECT DISTINCT e1.Salary
FROM Employee e1
WHERE 2 = (SELECT COUNT(DISTINCT e2.Salary)
FROM Employee e2
WHERE e2.Salary > e1.Salary)
```

```
Salary
------
55000
```

Meaning: a salary is the 3rd highest if exactly 2 distinct salaries are bigger than it. For the Nth: replace 2 by N-1.

Part 9 - Views

A **view** is a saved SELECT with a name - a **virtual table**. It stores the definition, not a separate copy of data. Query it like a table.

```
CREATE VIEW view_name AS
SELECT columns FROM table WHERE condition;
```

Build it in two steps: first solve the plain SELECT, then put `CREATE VIEW name AS` above it. "Create a view of students with marks above 80":

```
CREATE VIEW Top_Students AS
SELECT StudentID, Name, Department, Marks FROM Student WHERE Marks > 80;

SELECT * FROM Top_Students;                          -- using the view
SELECT Name FROM Top_Students WHERE Department='CSE';-- filter a view further
```

```
CREATE VIEW Student_Course_View AS                   -- a join view hides the join from users
SELECT s.StudentID, s.Name, c.CourseName, e.Marks
FROM Student s JOIN Enrollment e ON s.StudentID = e.StudentID
               JOIN Course c    ON e.CourseID  = c.CourseID;
```

Updating, replacing, dropping

```
UPDATE Top_Students SET Marks = 90 WHERE StudentID = 101;     -- changes the BASE table row
CREATE OR REPLACE VIEW Top_Students AS SELECT ... ;     -- change the definition (or ALTER VIEW)
DROP VIEW Top_Students;                                      -- removes only the view; table stays

CREATE VIEW CSE_Students AS
SELECT StudentID, Name, Department, Marks FROM Student WHERE Department = 'CSE'
WITH CHECK OPTION;       -- blocks changes through the view that would make the row leave the view
```

- **WITH CHECK OPTION**: UPDATE CSE_Students SET Department='ECE' ... is rejected because the new row would not satisfy Department='CSE'.
- **Not updatable** (cannot be updated through the view): views using GROUP BY, aggregates, DISTINCT, UNION. Reason: a view row like "CSE average = 78" does not correspond to one base row.
- **Uses:** simplify complex queries, hide columns/rows (security), give a stable interface. Not automatically faster.

> **EXAM ANSWER:** Define a view; syntax; one example; advantages (simplicity, security, abstraction/logical independence); limitations (not always updatable, depends on base tables); view vs table (virtual vs stored) vs CTE (one statement only).

Part 10 - Procedures, functions, triggers

These are "SQL programs stored in the database". Learn the **template** of each, then only the middle changes per question.

10.1 Why DELIMITER?

The body contains semicolons. The MySQL client would stop at the first one. `DELIMITER //` tells the client "my statement ends at //". Switch back with `DELIMITER ;`. DELIMITER is a client command, not SQL.

10.2 Stored procedure (you CALL it; can do multi-step work)

```
DELIMITER //
CREATE PROCEDURE ProcName ( [IN|OUT|INOUT] name TYPE, ... )
BEGIN
    -- statements
END //
DELIMITER ;
CALL ProcName(args);
```

Parameter mode	Meaning	Trigger words
IN (default)	value goes in	given a department / accept an ID
OUT	procedure sends a value back	return the count through a parameter
INOUT	goes in, modified, comes back	accept, modify and return

Example 1 - IN: students of a given department.

```
DELIMITER //
CREATE PROCEDURE GetStudents(IN p_dept VARCHAR(50))
BEGIN
    SELECT Name, Marks FROM Student WHERE Department = p_dept;   -- no quotes around p_dept!
END //
DELIMITER ;
CALL GetStudents('CSE');
```

Example 2 - OUT: count students into a variable.

```
DELIMITER //
CREATE PROCEDURE CountStudents(OUT p_total INT)
BEGIN
    SELECT COUNT(*) INTO p_total FROM Student;       -- INTO stores the result in the parameter
END //
DELIMITER ;
CALL CountStudents(@total);      -- @total = session variable that receives the value
SELECT @total;                   -- 6
```

Example 3 - UPDATE with two inputs: "raise a student's marks by a bonus".

```
DELIMITER //
CREATE PROCEDURE IncreaseMarks(IN p_id INT, IN p_bonus INT)
BEGIN
    UPDATE Student SET Marks = Marks + p_bonus WHERE StudentID = p_id;
END //
DELIMITER ;
CALL IncreaseMarks(102, 5);
```

Example 4 - IF inside a procedure:

```
DELIMITER //
CREATE PROCEDURE CheckResult(IN p_marks INT)
BEGIN
    IF p_marks >= 40 THEN
        SELECT 'PASS' AS Result;
    ELSEIF p_marks >= 35 THEN
        SELECT 'GRACE' AS Result;
    ELSE
        SELECT 'FAIL' AS Result;
    END IF;                                  -- END IF closes the IF; END closes the procedure
END //
DELIMITER ;
```

Manage: `SHOW PROCEDURE STATUS;` `SHOW CREATE PROCEDURE name;` `DROP PROCEDURE IF EXISTS name;`

10.3 Stored function (returns exactly ONE value; used inside SELECT)

```
DELIMITER //
CREATE FUNCTION FuncName ( param TYPE, ... )     -- parameters are inputs only
RETURNS return_type                              -- declares the type
DETERMINISTIC          -- same input -> same output (MySQL wants this, or NO SQL / READS SQL DATA)
BEGIN
    ...
    RETURN value;                                -- sends the value back
END //
DELIMITER ;
SELECT FuncName(args);
```

```
-- square of a number
CREATE FUNCTION SquareNumber(p_num INT) RETURNS INT DETERMINISTIC NO SQL
BEGIN  RETURN p_num * p_num;  END //

-- average of two marks (DECIMAL because average may have a fraction)
CREATE FUNCTION AverageMarks(p_a INT, p_b INT) RETURNS DECIMAL(5,2) DETERMINISTIC NO SQL
BEGIN  RETURN (p_a + p_b) / 2;  END //

-- percentage: guard against divide-by-zero
CREATE FUNCTION Percentage(p_obt INT, p_total INT) RETURNS DECIMAL(7,2) DETERMINISTIC NO SQL
BEGIN
    IF p_total <= 0 THEN RETURN NULL; END IF;
    RETURN 100.0 * p_obt / p_total;
END //

-- grade (use the thresholds given in the question!)
CREATE FUNCTION CalculateGrade(p_marks INT) RETURNS VARCHAR(2) DETERMINISTIC NO SQL
BEGIN
    IF    p_marks >= 90 THEN RETURN 'A+';
    ELSEIF p_marks >= 80 THEN RETURN 'A';
    ELSEIF p_marks >= 70 THEN RETURN 'B';
    ELSEIF p_marks >= 60 THEN RETURN 'C';
    ELSE RETURN 'F';
    END IF;
END //
-- use it:  SELECT Name, CalculateGrade(Marks) AS Grade FROM Student;
```

Procedure	Function
called with CALL	used inside an expression: SELECT f(x)
may do many things, can return result sets	must return ONE scalar value
IN, OUT, INOUT parameters	input parameters only
no RETURNS clause	RETURNS type is compulsory, body uses RETURN
can change data (DML) freely	usually used for calculation

TRAP: RETURNS (with S) declares the type in the header. RETURN (no S) sends the value inside the body. Mixing them up is a favourite syntax mistake.

10.4 Triggers (run AUTOMATICALLY on INSERT / UPDATE / DELETE)

Trigger words: "**whenever / automatically / before inserting / after deleting / log / audit / reject / prevent**". You never CALL a trigger.

```
DELIMITER //
CREATE TRIGGER trigger_name
{BEFORE | AFTER} {INSERT | UPDATE | DELETE} ON table_name
FOR EACH ROW                      -- runs once per affected row
BEGIN
    -- uses NEW.col and/or OLD.col
END //
DELIMITER ;
```

Event	OLD.col	NEW.col
INSERT	not available	the row being inserted
UPDATE	value before the change	value after the change
DELETE	row being deleted	not available

Wording	Timing + event	Uses
reject / validate / adjust before saving	BEFORE INSERT/UPDATE	NEW; SIGNAL to reject; SET NEW.col = ... to correct
prevent deletion	BEFORE DELETE	OLD.col in the condition + SIGNAL
record / log what happened	AFTER INSERT/UPDATE/DELETE	INSERT into a log table using NEW/OLD

Reject negative marks (BEFORE INSERT + SIGNAL):

```
DELIMITER //
CREATE TRIGGER reject_negative_marks
BEFORE INSERT ON Student
FOR EACH ROW
BEGIN
    IF NEW.Marks < 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Marks cannot be negative';
    END IF;
END //
DELIMITER ;
```

Cap marks at 100 (BEFORE UPDATE, correcting instead of rejecting):

```
CREATE TRIGGER cap_marks BEFORE UPDATE ON Student FOR EACH ROW
BEGIN
    IF NEW.Marks > 100 THEN SET NEW.Marks = 100; END IF;
END //
```

Prevent deleting high scorers (uses OLD):

```
CREATE TRIGGER protect_high BEFORE DELETE ON Student FOR EACH ROW
BEGIN
    IF OLD.Marks > 80 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Cannot delete high scorer';
    END IF;
END //
```

Audit log: first create the log table, then AFTER triggers insert into it. A single-statement body needs no BEGIN/END or delimiter change.

```
CREATE TABLE StudentLog (
    LogID INT AUTO_INCREMENT PRIMARY KEY, StudentID INT, Name VARCHAR(100),
    Action VARCHAR(20), OldMarks DECIMAL(5,2), NewMarks DECIMAL(5,2),
    ActionTime TIMESTAMP DEFAULT CURRENT_TIMESTAMP);

CREATE TRIGGER log_insert AFTER INSERT ON Student FOR EACH ROW
  INSERT INTO StudentLog(StudentID,Name,Action,NewMarks)
  VALUES (NEW.StudentID,NEW.Name,'INSERT',NEW.Marks);

CREATE TRIGGER log_update AFTER UPDATE ON Student FOR EACH ROW
  INSERT INTO StudentLog(StudentID,Name,Action,OldMarks,NewMarks)
  VALUES (OLD.StudentID,NEW.Name,'UPDATE',OLD.Marks,NEW.Marks);

CREATE TRIGGER log_delete AFTER DELETE ON Student FOR EACH ROW
  INSERT INTO StudentLog(StudentID,Name,Action,OldMarks)
  VALUES (OLD.StudentID,OLD.Name,'DELETE',OLD.Marks);
```

Manage: `SHOW TRIGGERS; DROP TRIGGER name;`. Pros: automatic validation/audit, central rules. Cons: hidden logic, harder debugging, extra write cost.

> **REMEMBER:** Procedure = CALL it. Function = use it in an expression, must return a value. Trigger = fires itself on a table event. INSERT trigger -> only NEW. DELETE trigger -> only OLD. UPDATE -> both.

Part 11 - Indexes, B-tree/B+ tree, assertions, query processing

11.1 Index

An index is an extra sorted structure (like a book's index) that lets the DBMS find rows without scanning the whole table.

```
CREATE INDEX idx_name ON Student(Name);
CREATE INDEX idx_dept_marks ON Student(Department, Marks);   -- composite: order of columns matters
CREATE UNIQUE INDEX idx_email ON Student(Email);
SHOW INDEX FROM Student;      DROP INDEX idx_name ON Student;
```

- Speeds up: equality, range, JOIN, ORDER BY, prefix LIKE 'An%'. Does not help LIKE '%an' (leading wildcard).
- Cost: extra space, slower INSERT/UPDATE/DELETE (index must be updated). Do not index everything.
- Composite (Department, Marks) helps queries filtering on Department (leftmost column), or Department AND Marks; not much for Marks alone.
- Types: primary (on the key), unique, secondary/non-unique, composite, clustered (rows stored in index order) vs non-clustered.

11.2 B-tree vs B+ tree

Point	B-tree	B+ tree
Where are data entries?	in internal AND leaf nodes	only in leaf nodes
Internal nodes	keys + data/pointers	only separator keys + child pointers
Leaf nodes linked?	no	YES - linked list of leaves
Search	may stop at an internal node	always goes down to a leaf
Range query (20 to 70)	slower, tree traversal	fast: find 20, walk the leaf links
Used in	some file systems	most DB indexes (InnoDB)

```
B+ tree:           [ 30 | 60 ]
                  /     |     \
        [10,20] -> [30,40,50] -> [60,70,80]      <- leaves linked left to right
```

Both are balanced (all leaves at the same depth) and have high fan-out, so the tree is short and a lookup needs only a few disk reads.

11.3 Assertion

An assertion is a condition the whole database must always satisfy (can span tables). Standard SQL: `CREATE ASSERTION salary_check CHECK (NOT EXISTS (SELECT * FROM Employee WHERE Salary < 0));` **MySQL does not support it**; use CHECK (row-level) or triggers.

11.4 Query processing and optimization (theory answer)

1. **Parsing and validation** - syntax, table/column existence, permissions.
2. **Translation** - into relational-algebra form.
3. **Optimization** - generate equivalent plans, estimate cost, choose the cheapest.
4. **Execution** - run the plan (scans, joins, filters).
5. **Return** the rows.

Technique	Idea
Selection pushdown	apply WHERE filters as early as possible to shrink data
Projection pushdown	keep only needed columns early
Index use	seek instead of scan
Join ordering	join small/selective inputs first
Join algorithms	nested-loop (for each outer row find inner matches), hash join (equality), merge join (sorted inputs)

Technique	Idea
Cost-based choice	uses statistics: row counts, selectivity, I/O, CPU. Estimates can be wrong

`EXPLAIN SELECT ...` shows the chosen plan, not the data. Large-scale RDBMS challenges: performance (indexes/partitioning/sharding), concurrency (locking/MVCC), availability (replication, failover), security (least privilege), backup (replication is not a backup - mistakes get replicated).

Part 12 - SQL practice bank (cover the answer first!)

Method: read, run the 6 steps from Part 0 on paper, write the query, then compare. Outputs are real. Questions 1-12 are basic, 13-24 intermediate, 25-30 hard.

Q1. Show names and marks of CSE students scoring above 75, highest first.

```
SELECT Name, Marks FROM Student WHERE Department='CSE' AND Marks > 75 ORDER BY Marks DESC
```

```
Name  | Marks
------+------
Anu   | 85
Meena | 78
```

Why: WHERE for both conditions (AND), ORDER BY ... DESC to sort.

Q2. Show students whose names start with A or end with a.

```
SELECT Name FROM Student WHERE Name LIKE 'A%' OR Name LIKE '%a'
```

```
Name
-----
Anu
Priya
Meena
Arjun
```

Why: LIKE with % wildcard; OR because either is enough.

Q3. Show students from Bangalore or Chennai with marks between 70 and 90.

```
SELECT Name, City, Marks
FROM Student
WHERE City IN ('Bangalore','Chennai') AND Marks BETWEEN 70 AND 90
```

```
Name  | City      | Marks
------+-----------+------
Anu   | Bangalore | 85
Ravi  | Chennai   | 72
Meena | Chennai   | 78
```

Why: IN for the list, BETWEEN is inclusive.

Q4. Total number of students and their average marks, rounded to 2 decimals.

```
SELECT COUNT(*) AS total, ROUND(AVG(Marks),2) AS avg_marks FROM Student
```

```
total | avg_marks
------+----------
6     | 79.83
```

Why: No GROUP BY: whole table is one group.

Q5. Number of students in each city.

```
SELECT City, COUNT(*) AS n FROM Student GROUP BY City ORDER BY n DESC
```

```
City      | n
----------+--
Chennai   | 2
Bangalore | 2
Mysore    | 1
Hosur     | 1
```

Why: "each" -> GROUP BY.

Q6. Departments whose average marks exceed 80.

```
SELECT Department, AVG(Marks) AS avg_marks
FROM Student
GROUP BY Department
HAVING AVG(Marks) > 80
```

```
Department | avg_marks
-----------+----------
ECE        | 89.5
```

Why: Condition on a group result -> HAVING.

Q7. Student names with their HOD.

```
SELECT s.Name, d.HOD FROM Student s JOIN Department d ON s.Department = d.Department
```

```
Name  | HOD
------+----------
Anu   | Dr. Kumar
Ravi  | Dr. Kumar
Priya | Dr. Priya
Kiran | Dr. Ravi
Meena | Dr. Kumar
Arjun | Dr. Priya
```

Why: Name is in Student, HOD in Department; link on Department.

Q8. Every department with its HOD and number of students, including departments with none.

```
SELECT d.Department, d.HOD, COUNT(s.StudentID) AS students
FROM Department d
  LEFT
  JOIN Student s ON d.Department = s.Department
GROUP BY d.Department, d.HOD
```

```
Department | HOD       | students
-----------+-----------+---------
CSE        | Dr. Kumar | 3
ECE        | Dr. Priya | 2
EEE        | Dr. Ravi  | 1
ME         | Dr. Anil  | 0
```

Why: "including none" -> LEFT JOIN from Department; COUNT(s.StudentID) not COUNT(*).

Q9. Name, course name and marks for every enrollment.

```
SELECT s.Name, c.CourseName, e.Marks
FROM Student s
  JOIN Enrollment e ON s.StudentID=e.StudentID
  JOIN Course c ON e.CourseID=c.CourseID
ORDER BY s.Name
```

```
Name  | CourseName      | Marks
------+-----------------+------
Anu   | DBMS            | 88
Anu   | Python          | 92
Arjun | Data Structures | 90
Kiran | Python          | 68
Meena | DBMS            | 81
Priya | DBMS            | 95
Priya | Data Structures | 89
Ravi  | DBMS            | 75
```

Why: Three tables via the bridge Enrollment.

Q10. Names of students NOT enrolled in course C02 (same pattern answers "never ordered", "no match").

```
SELECT s.Name
FROM Student s
WHERE NOT EXISTS (SELECT 1
FROM Enrollment e
WHERE e.StudentID = s.StudentID AND e.CourseID='C02')
```

```
Name
-----
Ravi
Priya
Meena
Arjun
```

Why: NOT EXISTS correlated on StudentID.

Q11. Students who scored above the overall average.

```
SELECT Name, Marks FROM Student WHERE Marks > (SELECT AVG(Marks) FROM Student)
```

```
Name  | Marks
------+------
Anu   | 85
Priya | 91
Arjun | 88
```

Why: Scalar subquery.

Q12. Number of courses each student has taken, only students with 2 or more.

```
SELECT s.Name, COUNT(*) AS courses
FROM Student s
  JOIN Enrollment e ON s.StudentID=e.StudentID
GROUP BY s.StudentID, s.Name
HAVING COUNT(*) >= 2
```

```
Name  | courses
------+--------
Anu   | 2
Priya | 2
```

Why: JOIN then GROUP BY student, HAVING on the count.

Q13. Average DBMS (C01) marks.

```
SELECT AVG(Marks) AS dbms_avg FROM Enrollment WHERE CourseID='C01'
```

```
dbms_avg
--------
84.75
```

Why: Enrollment.Marks is course marks; no join needed because CourseID is already in Enrollment.

Q14. Student(s) with the highest marks in DBMS, with name.

```
SELECT s.Name, e.Marks
FROM Student s
  JOIN Enrollment e ON s.StudentID=e.StudentID
WHERE e.CourseID='C01' AND e.Marks = (SELECT MAX(Marks)
FROM Enrollment
WHERE CourseID='C01')
```

```
Name  | Marks
------+------
Priya | 95
```

Why: Join for the name, scalar subquery for the max.

Q15. Courses with the number of enrolled students, including courses with none.

```
SELECT c.CourseName, COUNT(e.StudentID) AS enrolled
FROM Course c
  LEFT
  JOIN Enrollment e ON c.CourseID=e.CourseID
GROUP BY c.CourseID, c.CourseName
```

```
CourseName      | enrolled
----------------+---------
DBMS            | 4
Python          | 2
Data Structures | 2
AI              | 0
```

Why: LEFT JOIN from Course keeps AI with 0.

Q16. Each student with Pass (overall marks >= 75) / Fail.

```
SELECT Name, Marks, CASE WHEN Marks>=75 THEN 'Pass' ELSE 'Fail' END AS Result FROM Student
```

```
Name  | Marks | Result
------+-------+-------
Anu   | 85    | Pass
Ravi  | 72    | Fail
Priya | 91    | Pass
Kiran | 65    | Fail
Meena | 78    | Pass
Arjun | 88    | Pass
```

Why: CASE per row.

Q17. Department-wise count of students with grade A (>=85), B (70-84) and C (below 70).

```
SELECT Department, SUM(CASE WHEN Marks>=85 THEN 1 ELSE 0 END) AS A, SUM(CASE WHEN Marks>=70
    AND Marks<85 THEN 1 ELSE 0 END) AS B, SUM(CASE WHEN Marks<70 THEN 1 ELSE 0 END) AS C
FROM Student
GROUP BY Department
```

```
Department | A | B | C
-----------+---+---+--
CSE        | 1 | 2 | 0
ECE        | 2 | 0 | 0
EEE        | 0 | 0 | 1
```

Why: CASE inside SUM + GROUP BY gives a cross-tab.

Q18. Students scoring above their own department average.

```
SELECT s.Name
FROM Student s
WHERE s.Marks > (SELECT AVG(t.Marks)
FROM Student t
WHERE t.Department = s.Department)
```

```
Name
-----
Anu
Priya
```

Why: Correlated subquery on Department.

Q19. Same question with a window function (shows the average too).

```
WITH x AS (SELECT Name, Marks, AVG(Marks) OVER (PARTITION BY Department) AS dept_avg
FROM Student) SELECT Name, Marks, dept_avg
FROM x
WHERE Marks > dept_avg
```

```
Name  | Marks | dept_avg
------+-------+---------
Anu   | 85    | 78.33
Priya | 91    | 89.5
```

Why: Window in CTE, then filter outside.

Q20. Employees with the second-highest salary (with their names).

```
SELECT EmpName, Salary
FROM Employee
WHERE Salary = (SELECT MAX(Salary)
FROM Employee
WHERE Salary < (SELECT MAX(Salary)
FROM Employee))
```

```
EmpName | Salary
--------+-------
Asha    | 60000
Ravi    | 60000
```

Why: Max below the max. Or DENSE_RANK() = 2.

Q21. Top 2 earners in each employee department (exactly 2, ties by EmpID).

```
WITH r AS (SELECT EmpName, Dept, Salary, ROW_NUMBER() OVER (PARTITION BY Dept ORDER BY
    Salary DESC, EmpID) AS rn
FROM Employee) SELECT EmpName, Dept, Salary
FROM r
WHERE rn <= 2
```

```
EmpName | Dept  | Salary
--------+-------+-------
Kumar   | HR    | 50000
John    | HR    | 45000
Sita    | IT    | 75000
Asha    | IT    | 60000
Mary    | Sales | 55000
```

Why: ROW_NUMBER for exactly N.

Q22. Employees who earn more than their manager.

```
SELECT e.EmpName, e.Salary, m.EmpName AS Manager, m.Salary AS MgrSalary
FROM Employee e
  JOIN Employee m ON e.ManagerID=m.EmpID
WHERE e.Salary > m.Salary
```

```
EmpName | Salary | Manager | MgrSalary
--------+--------+---------+----------
Asha    | 60000  | Kumar   | 50000
Ravi    | 60000  | Kumar   | 50000
Sita    | 75000  | Asha    | 60000
Mary    | 55000  | Kumar   | 50000
```

Why: Self join.

Q23. Employees with no manager.

```
SELECT EmpName FROM Employee WHERE ManagerID IS NULL
```

```
EmpName
-------
Kumar
```

Why: IS NULL, not = NULL.

Q24. Departments (Employee.Dept) having more than one employee, with the department total salary.

```
SELECT Dept, COUNT(*) AS n, SUM(Salary) AS total
FROM Employee
GROUP BY Dept
HAVING COUNT(*) > 1
```

```
Dept | n | total
-----+---+-------
HR   | 2 | 95000
IT   | 3 | 195000
```

Why: GROUP BY + HAVING.

Q25. Salaries that appear more than once (duplicates).

```
SELECT Salary, COUNT(*) AS c FROM Employee GROUP BY Salary HAVING c > 1
```

```
Salary | c
-------+--
60000  | 2
```

Why: Classic duplicate finder.

Q26. Department with the highest total salary.

```
SELECT Dept, SUM(Salary) AS total FROM Employee GROUP BY Dept ORDER BY total DESC LIMIT 1
```

```
Dept | total
-----+-------
IT   | 195000
```

Why: ORDER BY aggregate alias + LIMIT 1.

Q27. Each student with their overall rank (1 = highest marks), dense.

```
SELECT Name, Marks, DENSE_RANK() OVER (ORDER BY Marks DESC) AS pos FROM Student ORDER BY pos
```

```
Name  | Marks | pos
------+-------+----
Priya | 91    | 1
Arjun | 88    | 2
Anu   | 85    | 3
Meena | 78    | 4
Ravi  | 72    | 5
Kiran | 65    | 6
```

Why: OVER with ORDER BY only (no PARTITION).

Q28. Students who are enrolled in BOTH C01 and C02.

```
SELECT s.Name
FROM Student s
WHERE s.StudentID IN (SELECT StudentID
FROM Enrollment
WHERE CourseID='C01') AND s.StudentID IN (SELECT StudentID
FROM Enrollment
WHERE CourseID='C02')
```

```
Name
----
Anu
```

Why: Two IN subqueries. (Alternative: GROUP BY StudentID HAVING COUNT(DISTINCT CourseID)=2 over C01,C02.)

Q29. Students enrolled in C01 but NOT in C02.

```
SELECT s.Name
FROM Student s
WHERE EXISTS (SELECT 1
FROM Enrollment e
WHERE e.StudentID=s.StudentID AND e.CourseID='C01') AND NOT EXISTS (SELECT 1
FROM Enrollment e
WHERE e.StudentID=s.StudentID AND e.CourseID='C02')
```

```
Name
-----
Ravi
Priya
Meena
```

Why: EXISTS and NOT EXISTS together.

Q30. For each department, the name of the student who has the highest marks, plus how far each student is below their dept topper.

```
SELECT Name, Department, Marks, MAX(Marks) OVER (PARTITION BY Department) - Marks AS gap
FROM Student
ORDER BY Department, gap
```

```
Name  | Department | Marks | gap
------+------------+-------+----
Anu   | CSE        | 85    | 0
Meena | CSE        | 78    | 7
Ravi  | CSE        | 72    | 13
Priya | ECE        | 91    | 0
Arjun | ECE        | 88    | 3
Kiran | EEE        | 65    | 0
```

Why: MAX as a window gives the group max beside every row; subtract.

Extra (write on paper): stored-program questions

Question	Skeleton to produce
Procedure to display students of a department given as input	CREATE PROCEDURE p(IN d VARCHAR(50)) BEGIN SELECT ... WHERE Department = d; END
Procedure returning count through OUT	OUT p_total INT ... SELECT COUNT(*) INTO p_total FROM Student;
Function returning PASS/FAIL	RETURNS VARCHAR(4) ... IF p_marks >= 40 THEN RETURN 'PASS'; ELSE RETURN 'FAIL'; END IF;
Trigger to log deleted students	AFTER DELETE ... INSERT INTO log VALUES (OLD....)
Trigger to stop negative marks	BEFORE INSERT ... IF NEW.Marks < 0 THEN SIGNAL SQLSTATE '45000' ...

Part 13 - Transactions and ACID

13.1 Definition

A **transaction** is a group of database operations treated as ONE logical unit of work: all happen or none happen. Operations: **R(X)** read item X, **W(X)** write item X, **COMMIT** make permanent, **ABORT/ROLLBACK** undo.

Standard example - bank transfer: A = 10,000, B = 5,000, transfer 2,000 from A to B.

```
R(A); A = A - 2000; W(A);     -- A = 8000
R(B); B = B + 2000; W(B);     -- B = 7000
COMMIT;                        -- total still 15,000
```

If a crash happens after debiting A but before crediting B, money vanishes. Atomicity forces a rollback.

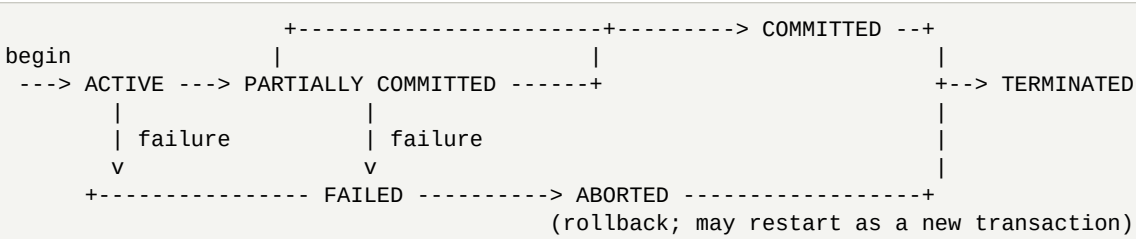
13.2 ACID - write all four with the transfer example

Property	Definition	In the transfer
Atomicity	All operations happen or none do	debit and credit both happen or both are undone
Consistency	A transaction takes the DB from one valid state to another (rules/constraints hold)	A + B total stays 15,000
Isolation	Concurrent transactions do not interfere; result as if run one after another	another transaction does not see A debited but B not yet credited
Durability	Once committed, changes survive crashes	after COMMIT, power failure does not lose the transfer

Who ensures what: Atomicity + Durability -> recovery manager (logs). Isolation -> concurrency control (locks/timestamps). Consistency -> programmer + constraints + the other three.

13.3 Transaction states (write all six + draw the diagram)

State	Meaning
Active	executing reads/writes
Partially committed	last statement executed, commit not yet durable
Committed	successfully completed, changes permanent
Failed	normal execution cannot continue
Aborted	rolled back, DB restored to the state before the transaction
Terminated	left the system (after commit or abort)



13.4 Why concurrency control is needed: the four problems

Problem	What happens
Lost update	T1 and T2 both read the old value; the later write overwrites the other's update
Dirty read	T2 reads a value written by T1 that is not committed and may be rolled back
Non-repeatable read	T1 reads a row twice and gets different values because T2 updated it in between
Inconsistent analysis (incorrect summary)	a sum is computed while another transaction is half-way through a transfer

Lost update numbers: X = 1000. T1 reads 1000, T2 reads 1000. T1 writes 1000-100 = 900. T2 writes 1000+50 = 1050. Final 1050 instead of the correct 950.

SQL isolation levels (weakest to strongest): READ UNCOMMITTED (dirty reads possible) -> READ COMMITTED -> REPEATABLE READ (MySQL InnoDB default) -> SERIALIZABLE.

Part 14 - Schedules, serializability, recoverability

14.1 Vocabulary

- **Schedule** = an ordering of operations from several transactions that keeps each transaction's own order.
- **Serial**: T1 completely, then T2 (no interleaving). **Non-serial**: operations interleaved.
- **Serializable**: equivalent in effect to some serial schedule.
- Notation: R1(A) = T1 reads A. W2(B) = T2 writes B. C1 = T1 commits. A1 = T1 aborts.

14.2 When do two operations CONFLICT?

> **REMEMBER:** Conflict = **different transactions** AND **same data item** AND **at least one is a write**. So R-R never conflicts. R-W, W-R, W-W do. Operations on different items never conflict.

14.3 Precedence-graph test for conflict serializability (the method - follow it every time)

1. Draw one **node per transaction**.
2. Go through the schedule left to right. For each item, for every pair of operations (earlier Ti op, later Tj op) that CONFLICT, draw an arrow **Ti -> Tj**.
3. Ignore read-read pairs and pairs inside the same transaction.
4. If the graph has a **cycle** -> NOT conflict-serializable. No cycle -> conflict-serializable.
5. If no cycle, any topological order of the nodes is an equivalent serial schedule.

Worked A: R1(A) W1(A) R2(A) W2(A) R1(B) W1(B) C1 C2

- Item A: R1(A) before W2(A) -> T1->T2; W1(A) before R2(A) -> T1->T2; W1(A) before W2(A) -> T1->T2.
- Item B: only T1 touches B -> no edge.
- Graph: T1 -> T2, no cycle -> conflict-serializable, equivalent serial order T1, T2.

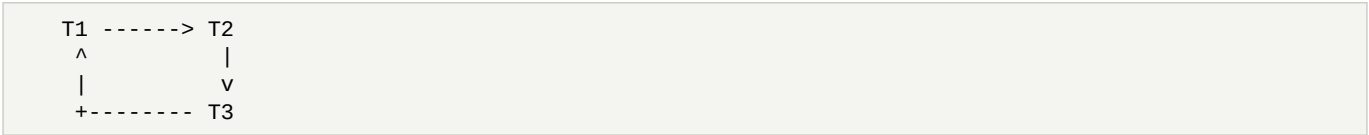
Worked B (lost update): R1(A) R2(A) W1(A) W2(A) C1 C2

Conflicting pair	Edge
R1(A) ... W2(A)	T1 -> T2
R2(A) ... W1(A)	T2 -> T1
W1(A) ... W2(A)	T1 -> T2

Edges T1->T2 and T2->T1 form a cycle -> NOT conflict-serializable.

Worked C (three transactions): W1(A) R2(A) W2(B) R3(B) W3(C) R1(C)

- W1(A) before R2(A): T1->T2.
- W2(B) before R3(B): T2->T3.
- W3(C) before R1(C): T3->T1.
- Cycle T1->T2->T3->T1 -> NOT conflict-serializable.



> **TRAP:** Do not confuse the two graphs. **Precedence graph** (conflicts between operations) -> cycle means not conflict-serializable. **Wait-for graph** (who waits for whose lock) -> cycle means deadlock.

14.4 Recoverability classes

First find **read-from** dependencies: if T2 reads the value last written by T1, T2 depends on T1. (Scan backwards from the read to the most recent write on that item.)

Class	Rule
Recoverable	a transaction commits only AFTER every transaction it read from has committed
Cascadeless (ACA)	a transaction reads an item only after the writer has COMMITTED (no dirty reads)
Strict	no transaction reads OR writes an item until the previous writer has committed/aborted (no dirty reads or dirty writes)

Hierarchy: **Strict** $\subset$ **Cascadeless** $\subset$ **Recoverable** (strict implies cascadeless implies recoverable; not the reverse). Serializability is a separate property.

Schedule	Verdict and reason
W1(A) R2(A) C1 C2	Recoverable (C1 before C2). Not cascadeless (R2 before C1: dirty read). Not strict.
W1(A) R2(A) C2 A1	NON-recoverable: T2 read T1's value and committed, then T1 aborted.
W1(A) C1 R2(A) W2(A) C2	Strict, cascadeless, recoverable: T2 accesses A only after C1.
W1(A) W2(A) C1 C2	Recoverable and cascadeless (no reads) but NOT strict (W2 overwrote uncommitted W1: dirty write).

Cascading rollback: W1(A) R2(A) W2(B) R3(B) A1 -> T2 read T1's uncommitted A so T2 must roll back; T3 read T2's B so T3 must roll back too. Cascadeless schedules prevent this chain.

Counter-example: W1(A) R2(A) C2 C1 has the single edge T1->T2 (conflict-serializable) but is not recoverable because T2 commits before T1.

How to classify a schedule (answer checklist)

1. Serial or not? (do operations of different transactions interleave?)
2. Conflict-serializable? Build precedence graph.
3. Recoverable? List read-from pairs; check commit order.
4. Cascadeless? Any read of uncommitted data?
5. Strict? Any read OR write of an uncommitted item?

14.5 Locking and Two-Phase Locking (2PL)

Held \ Requested	S (shared, read)	X (exclusive, write)
S	compatible	NOT compatible
X	NOT compatible	NOT compatible

- **S lock**: many readers allowed together. **X lock**: one writer, nobody else reads or writes.
- **2PL rule**: a transaction has a **growing phase** (only acquire locks) and then a **shrinking phase** (only release). The moment it releases any lock, it may not acquire a new one. The point where it holds its last lock = lock point.
- 2PL guarantees conflict serializability, but **does not prevent deadlock** and can allow cascading rollback.

Check: LOCK-X(A) R(A) W(A) LOCK-X(B) R(B) W(B) UNLOCK(A) UNLOCK(B) -> follows 2PL. LOCK-X(A) R(A) UNLOCK(A) LOCK-X(B) ... -> violates 2PL (acquires after releasing).

Variant	Extra rule	Gain
Basic 2PL	no lock after the first release	conflict serializability
Strict 2PL	hold all X (write) locks until commit/abort	no dirty reads/writes, strict schedules, easier recovery
Rigorous 2PL	hold ALL locks (S and X) until commit/abort	serial order = commit order

> **REMEMBER:** Strict 2PL still allows deadlock. "Strict" refers to holding write locks, not to avoiding deadlock.

14.6 Timestamp ordering

- Each transaction T gets a unique timestamp TS(T) when it starts. Smaller = older.
- Each item X keeps **RT(X)** = largest timestamp of any transaction that read it, **WT(X)** = largest timestamp of any that wrote it.

Request by T	If this holds -> reject (T rolls back, restarts with new timestamp)	Else
READ(X)	TS(T) < WT(X) (value was written by a younger transaction)	allow; RT(X) = max(RT(X), TS(T))
WRITE(X)	TS(T) < RT(X) OR TS(T) < WT(X)	allow; WT(X) = TS(T)

Example 1: RT(X)=0, WT(X)=0, TS(T1)=5, TS(T2)=10. T2 writes X: allowed, WT(X)=10. T1 then reads X: 5 < 10 -> reject T1.

Example 2: TS(T)=12, RT(X)=15, WT(X)=10. T reads X: 12 >= 10 OK (RT stays 15). T writes X: 12 < 15 -> reject.

- Pros: no locks -> no deadlock. Cons: many rollbacks, possible starvation, basic version not recoverability-safe. Thomas write rule (ignore obsolete write) is a variant - use only if the question says so.

14.7 Optimistic control and MVCC

- **Optimistic**: 3 phases - **read** (work on private copies), **validation** (check for conflicts), **write** (publish if valid, otherwise restart). Good when conflicts are rare.
- **MVCC** (multi-version): keep several versions of each item; a reader sees the version valid for its snapshot, so readers do not block writers. Costs storage and cleanup. Used by MySQL InnoDB and PostgreSQL.

14.8 Deadlock

Deadlock = a cycle of transactions each waiting for a lock held by another; none can proceed.

Step	Action
1	T1 locks X(A)
2	T2 locks X(B)
3	T1 requests X(B) -> waits for T2
4	T2 requests X(A) -> waits for T1

Wait-for graph: T1 -> T2 and T2 -> T1 = cycle = deadlock.

Detection and recovery

1. Build wait-for graph (node per transaction; Ti -> Tj if Ti waits for a lock held by Tj).
2. If there is a cycle, pick a **victim** (least work done / cheapest rollback / youngest).
3. Roll back the victim, release its locks, others continue; restart victim later.
4. Avoid choosing the same victim repeatedly (starvation).

Prevention

- **Ordered locking**: everybody requests locks in one global order (A before B) -> no cycle possible.
- **Wait-die** (non-preemptive) and **wound-wait** (preemptive) use timestamps:

Situation	Wait-die	Wound-wait
OLDER requests lock held by YOUNGER	older WAITS	older WOUNDS (aborts) the younger
YOUNGER requests lock held by OLDER	younger DIES (aborts)	younger WAITS

Mnemonic: in wait-die the *old* may wait. In wound-wait the *old* is aggressive. Restarted transactions keep their original timestamp. Example TS(T1)=10 (older), TS(T2)=20: T1 requests T2's lock -> wait-die: wait; wound-wait: abort T2. T2 requests T1's lock -> wait-die: T2 aborts; wound-wait: T2 waits.

Deadlock vs starvation: deadlock = circular wait, nobody progresses. Starvation = one transaction repeatedly delayed/restarted while others progress.

14.9 Recovery

- Failure types: **transaction** (error, deadlock victim), **system** (crash, power loss; memory lost), **media** (disk loss; needs backup + log).
- Log records: <T START>, <T, X, old, new> (update), <T COMMIT>, <T ABORT>.
- **Write-Ahead Logging (WAL)**: the log record must reach stable storage BEFORE the changed data page is written; the commit record must be durable before reporting commit.

```
<T1 START>
<T1, A, 10000, 8000>      -- item, OLD value, NEW value
<T1, B, 5000, 7000>
<T1 COMMIT>
```

Action	Uses	For
UNDO	old values	transactions that did NOT commit before the crash (restore old values, in reverse order)
REDO	new values	transactions that DID commit (re-apply in forward order)

Crash examples: crash after A=8000 written but no COMMIT -> UNDO: A back to 10,000. Crash after COMMIT but pages not yet on disk -> REDO: A=8000, B=7000.

Deferred update: DB updated only after commit (no UNDO needed; REDO if needed). **Immediate update**: DB may be updated before commit (needs UNDO and REDO).

Checkpoint: periodically records active transactions/dirty pages and forces logs/pages so recovery starts from the checkpoint instead of the beginning of the log. A checkpoint is not a backup.

Shadow paging: keep a shadow page table (old) and a current table. Updates are written to new pages. Commit = atomically switch the root pointer to the current table. Crash before commit = discard current, shadow is intact. No UNDO/REDO log needed; cost: page copying, fragmentation, garbage collection, hard concurrency.

Solved log question: <T1 START> <T1,A,100,150> <T1 COMMIT> <T2 START> <T2,B,200,180> CRASH. -> T1 committed: REDO A=150. T2 not committed: UNDO B=200.

Part 15 - Theory answer frames and cheat sheet

15.1 What to include for common long answers

Question	Structure that earns marks
Explain ACID	define transaction; the 4 properties; link each to the transfer example; what ensures it (recovery manager, concurrency control)
Transaction states	six states, one line each, diagram, failure paths, restart option
Explain 2PL	S/X lock compatibility; growing/shrinking; lock point; example; guarantees serializability; deadlock still possible; strict/rigorous
Conflict serializability problem	state conflict rule; draw graph with edges per conflicting pair; cycle test; give serial order if acyclic
Deadlock	definition; 4-step example; wait-for graph; detection (cycle, victim); prevention (ordering, wait-die/wound-wait); starvation
Recovery	failure types; log format; WAL; UNDO/REDO; checkpoint; backup vs replication; shadow paging if asked
B-tree vs B+ tree	table of where data lives, leaf links, search, range queries; small diagram
Query optimization	5 processing stages; pushdowns; indexes; join order/algorithm; cost estimation caveat
View / Trigger / Procedure	definition, syntax, example, advantages, limitations, comparison with the neighbour concept

15.2 One-page cheat sheet - SQL templates

```
FILTER      SELECT cols FROM t WHERE cond ORDER BY col DESC LIMIT n;
GROUP       SELECT g, AGG(x) FROM t [WHERE row] GROUP BY g [HAVING AGG(x) cond];
INNER JOIN  SELECT .. FROM a JOIN b ON a.k = b.k [JOIN c ON b.k2 = c.k2];
LEFT JOIN   SELECT .. FROM a LEFT JOIN b ON a.k = b.k;   unmatched: WHERE b.key IS NULL
COUNT zero  COUNT(b.key) after LEFT JOIN (not COUNT(*))
SELF JOIN   FROM emp e JOIN emp m ON e.ManagerID = m.EmpID
ABOVE AVG   WHERE x > (SELECT AVG(x) FROM t)
OWN-GROUP   WHERE x > (SELECT AVG(x) FROM t t2 WHERE t2.g = t.g)
EXISTS      WHERE [NOT] EXISTS (SELECT 1 FROM b WHERE b.k = a.k)
CASE        CASE WHEN c1 THEN v1 WHEN c2 THEN v2 ELSE v3 END AS name
COUNT-BY    SUM(CASE WHEN cond THEN 1 ELSE 0 END)
WINDOW      AVG(x) OVER (PARTITION BY g)      RANK() OVER (PARTITION BY g ORDER BY x DESC)
TOPPER      WITH r AS (SELECT .., RANK() OVER (PARTITION BY g ORDER BY x DESC) rnk FROM t)
            SELECT .. FROM r WHERE rnk = 1;
NTH HIGHEST SELECT DISTINCT x FROM t ORDER BY x DESC LIMIT 1 OFFSET n-1;
VIEW        CREATE VIEW v AS SELECT ...;        DROP VIEW v;
PROCEDURE   DELIMITER // CREATE PROCEDURE p(IN a INT) BEGIN ... END // DELIMITER ;  CALL p(1);
FUNCTION    CREATE FUNCTION f(a INT) RETURNS INT DETERMINISTIC BEGIN RETURN a*a; END
TRIGGER     CREATE TRIGGER t BEFORE|AFTER INSERT|UPDATE|DELETE ON tbl FOR EACH ROW BEGIN .. END
```

15.3 Last-hour rules (the mistakes that cost marks)

- WHERE filters rows, HAVING filters groups. Aggregates never in WHERE.
- LEFT JOIN + COUNT(right key). Filter on the right table goes in ON, not WHERE.
- Above overall average = scalar subquery. Above own group average = correlated (or window).
- Rank filter needs CTE. ROW_NUMBER = exactly N, RANK = ties/gaps, DENSE_RANK = ties/no gaps.
- CASE: strictest WHEN first, END is compulsory, alias after END.
- Function: RETURNS type + RETURN value. Procedure: CALL. Trigger: INSERT->NEW, DELETE->OLD, UPDATE->both.
- Never UPDATE/DELETE without WHERE. NULL needs IS NULL. Text in single quotes.
- Conflict = different txn + same item + at least one write. Cycle in precedence graph = not conflict-serializable.
- Recoverable (commit order) <- Cascadeless (no dirty reads) <- Strict (no dirty reads or writes).
- 2PL: all locks before any release; can still deadlock; strict 2PL holds X locks till end. Wait-die: old waits. Wound-wait: old kills young.
- UNDO = old values for uncommitted. REDO = new values for committed. WAL = log before data page.